Data Structure
Adnan Salman **Assignment 5**
==========

Objectives: To gain more experience with list and stacks

Procedure:

1. In this assignment, you will use a linked list of stacks to decode a message. The list must be doubly linked and circular. Use the following listNodeType definition:

```
struct listNodeType {
   Stack* stk;
   listNodeType* back;
   listNodeType* next;
};
```

2. The doubly linked list interface you need to support is defined by:

```
class DLCList {
public:
   DLCList(int n = 6 );
   ~DLCList(){};

   bool    deleteCurrent();
   Stack*  getCurrentStack() const;
   void    goBackward(int nNodes);
   void    goForward(int nNodes);
   bool    isEmpty();
private:
   listNodeType* cur;
};
```

3. Copy to your working directory the template files ArrayStackT.h and StackADTT.h provided in this assignment.

4. Your program is going to define and manipulate a list of stacks. The following detail the requirements of the various methods in DLCList class:

   Constructor:

   The constructer you are to use takes a number of list elements to generate. This constructor is to dynamically allocate n listNodes, linking them together into a doubly-linked circular list. For each listNode that gets dynamically allocated, set its "stk" field to be a new dynamically allocated (initially empty) stack. After the constructor is finished, you will have n dynamically allocated stacks. One stack is stored in each of the n listNode elements. Set "cur" to point to the head of the list of stacks. (At this point it doesn't matter which of the listNode it points to since they are all the same)

   Destructor:
   The destructor is to traverse the list, starting at "cur" (if it is non-NULL). At each node, delete first the stack, and then delete the listNode itself.

goBackward:
Advance "cur" the appropriate number of spots using the back pointer.

goForward :
Advance "cur" the appropriate number of spots using next pointer. For example, if there are 10 elements in the list and "goBackward(3)" is executed, "cur" winds up at "cur-> prev -> prev->prev". Invoking "goForward(273)" is also valid; it is interpreted by following "next" pointer 273 times, since the list is circular, no problem arises.

getCurrentStack:
returns the pointer to the stack in the node currently referenced by "cur". If "cur" is NULL (i.e. if the list of stacks is empty), then return NULL.

deleteCurrent:
deletes the node referenced by current. If "cur" is NULL, return FALSE. Otherwise your implementation must first use "delete" on the Stack pointer in "cur->stk" so that the stack can be returned to free storage. Then, reroute pointers in the usual way. Re-set "cur" to point to the listNode which previously followed "cur". If this was the last node in the list, then set "cur" to NULL.

5. You will write a main program to use these interfaces as follows. You will be reading a file (provided) with a message encoded in it. The first item in the file is the value of the integer to be used in the DLCList constructor. Remaining entries in the file will be one of the following:

-n	invoke goBackward(n)
n	invoke goForward(n)
0 P char	push char on the stack in the current list node
0 D	use "deleteCurrent" to delete the current listNode and its stack

   Notice that this means the loop in your program will first read an integer. If it's negative or positive, it simply invokes the appropriate method. If it's 0, it reads additional parameters.

6. A sample input file would be:

   6
   0 P n 4 0 P e -5 0 P e -1 0 P t 4 0 P s
   2 0 P e 0 P l 0 P p 1 0 P h 3 0 P i
   -1 0 D -1 0 P o 0 P i 0 P s 3 0 P m 0 P o -1 0 D 2 0 P s 0 P i -1 0 P t -1 0 P c 2 0 P m 3

   You are to read and process these directives until end of file. When end of file is encountered, begin with stack in "cur" , pop and print its characters until the stack is empty. Output a single blank. Delete that listNode (with "deleteCurrent") and repeat the process until the list of stacks has been emptied.

7. For the example file above, the output would be: _____